# AI Invoice Assistant – AI development log

Claude Code (CLI) with Claude Opus · 7 of 12 episodes; full history in the git log

**Working rule** (CLAUDE.md): the agent shows each function and writes it only after the user's OK; tests and commits on request.

### 1 · First AI feature: customer chat (Ep. 2–3) modified

| | |
|---|---|
| **Task / context** | A bot that creates, changes and deletes customers by chat; llama3.2:3b, "the chat never saves". |
| **Agent's proposal** | Text commands parsed by Python; all customer data sent to the bot so it could choose. |
| **Tools** | File edits after approval, Python, Ollama, chat tests with the real bot. |
| **Verification** | 8 chat tests: 2/8. |
| **Outcome** | Changed: Python looks records up and decides, the bot only extracts the request → 7/8. |
| **Lesson** | The small model must not hold data or make decisions – the basis of the whole design. |

### 2 · Checker bots and prompt tuning (Ep. 3–4) rejected – failed

| | |
|---|---|
| **Task / context** | Make the bot more reliable by letting a second bot judge or correct its answers. |
| **Agent's proposal** | Judge and coach bots, a correction loop, spaCy name recognition, 21 tuning experiments by a sub-agent. |
| **Tools** | File edits, a sub-agent, many bot runs, package install. |
| **Verification** | Hold-out and fresh scenarios: the checker broke correct answers and invented placeholders; scores on known cases were too optimistic. |
| **Outcome** | Rejected; only a simpler, general prompt was kept. |
| **Lesson** | Another small model adds errors; checks belong in deterministic Python. Test on unseen wordings. |

### 3 · Products and invoices by a specialised agent (Ep. 5) accepted

| | |
|---|---|
| **Task / context** | Build two more entities following the customer pattern. |
| **Agent's proposal** | An "entity-builder" agent description, then code, endpoints and bots for products and invoices. |
| **Tools** | A sub-agent with file and command access in the project. |
| **Verification** | End-to-end API test 30/30; bot weaknesses recorded as open problems. |
| **Outcome** | Accepted. |
| **Lesson** | A written-down method makes the agent fast and consistent – it also copies the method's weaknesses. |

### 4 · One chat with forms (Ep. 7–8) modified

| | |
|---|---|
| **Task / context** | One chat for everything, turnover questions in plain English. |
| **Agent's proposal** | A router bot, an assistant with tools, pickers in the website, Python guards against invented records. |
| **Tools** | File edits incl. the website, bot runs, an acceptance-tester agent. |
| **Verification** | Repeated tests: the model copied ids wrong and dropped data in turnover answers. |
| **Outcome** | Changed: structured input goes through forms, and Python answers the turnover forms itself. |
| **Lesson** | Where input can be structured, a form beats asking the model to extract it. |

### 5 · Test scenarios by a test-writing agent (Ep. 10) accepted

| | |
|---|---|
| **Task / context** | The project's own scenario tests, 5 runs per scenario. |
| **Agent's proposal** | A "test-writer" agent and one test script per group of scenarios. |
| **Tools** | May write only in tests/ and run them – never changes the code under test. |
| **Verification** | First run: 12 failing scenarios, 7 after fixes. |
| **Outcome** | Accepted; remaining failures documented, bots not tuned to the test sentences. |
| **Lesson** | Separating "who writes tests" from "who writes code" keeps the tests honest. |

### 6 · Course requirements and evaluation (Ep. 11) accepted

| | |
|---|---|
| **Task / context** | Replaceable local model, failure handling, health check, container, evaluation with a baseline. |
| **Agent's proposal** | OpenAI-compatible API with configuration by environment, clear 503 errors, /health, a switch to run the bots without Python. |
| **Tools** | File edits after approval, tests with the model server off, the evaluation script. |
| **Verification** | Model server down → 503, data unchanged; bot alone 12/20, bot + Python 20/20. |
| **Outcome** | Accepted. |
| **Lesson** | A baseline makes the value of the Python layer measurable – and must be rerun after every change. |

### 7 · Code review before the submission partly accepted

| | |
|---|---|
| **Task / context** | Read all code with the agent, remove dead code, fix design weaknesses. |
| **Agent's proposal** | Several refactorings, incl. restricting the assistant to questions – predicted to break nothing. |
| **Tools** | File edits after approval, full test runs, comparison with older commits, a separate test server. |
| **Verification** | Against the last commit, 4 scenarios got worse. |
| **Outcome** | The refactorings were discarded and the tested version kept; two small router fixes were accepted after the user tested them. |
| **Lesson** | The agent states predictions confidently; only tests against a known baseline show whether they hold. |